

Detecting Intent Drift in Continuous Conversation via Temporal Transition Accumulation

Yue Wang¹, Dehang Fu¹, Jie Tan¹, Junxiao Han², Yao Wan³, Lixin Cui¹, Lu Bai^{4†}, Philip S. Yu, *Fellow, IEEE*⁵

¹School of Information, Central University of Finance and Economics, China

²School of Computer and Computing Science, Hangzhou City University, China

³College of Computer Science and Technology, Huazhong University of Science and Technology, China.

⁴School of Artificial Intelligence, Beijing Normal University, China

⁵Department of Computer Science, University of Illinois Chicago, USA

{wangyuecs, dehangfu, jietancs, cuilixin}@cufe.edu.cn, hanjx@hzc.edu.cn, wanyao@hust.edu.cn, bailu@bnu.edu.cn, psyu@uic.edu

Abstract—As large language models (LLMs)-driven conversational systems have advanced, users have become accustomed to engaging in standalone, continuous interactions. In such interactions, users may abruptly change their intent across turns. However, most existing intent detection methods focus on accurately extracting intents and slots from individual utterances, without considering the broader conversational dynamics. This makes them ill-equipped to handle long, evolving conversations where arbitrary intent drift can occur. To address this challenge, we define the intent drift detection task in continuous conversations. We then propose a differentiable method, termed *DriftHunter*, that enables neural networks to understand how user intent shifts as the conversation progresses via dynamically accumulating the temporal transition across turns. Unlike existing methods, our proposed method incrementally captures global and local transition patterns between intents and slots without relying on prior statistical results. Moreover, our model sequentially accumulates transition patterns across conversation turns. This allows it to learn temporal accumulated dynamics, enabling neural network models to better focus on the most trending user intents during continuous interaction. Experimental evaluations on real-world datasets demonstrate that the proposed method outperforms state-of-the-art baselines in both intent drift detection, intent identification, and slot-filling downstream tasks. Our case study analysis reveals that the learned temporal transition patterns explain the predicted intent drifts.¹

Index Terms—Intent drift detection, standalone-continuous conversation, and temporal transition accumulation

I. INTRODUCTION

The powerful capabilities of large language models (LLMs) on natural language understanding and generation [1] enable question answering (QA) systems [2] and chatbots [3] to facilitate more natural and human-like interactions. As a result, users accustomed to continuous single-session conversations [4], and often perceive these systems as genuine human customer service agents. This perception frequently leads users to overlook the necessity of manually switching sessions when engaging with distinct intents.

To provide accurate responses, QA and chatbot systems utilize various intent identification techniques to discern users'

intents or underlying goals from their utterances [5]–[7]. Such methods are crucial for both types of systems, as accurately inferring user intent enables a deeper understanding of user needs and facilitates the generation of contextually appropriate responses [8]. We argue that the existing approaches, which rely on static conversational histories for intent inference [9], are limited in their capacity to handle dynamically evolving intents in multi-turn conversations [10]. This limitation becomes particularly pronounced when users shift their intents without providing explicit lexical cues.

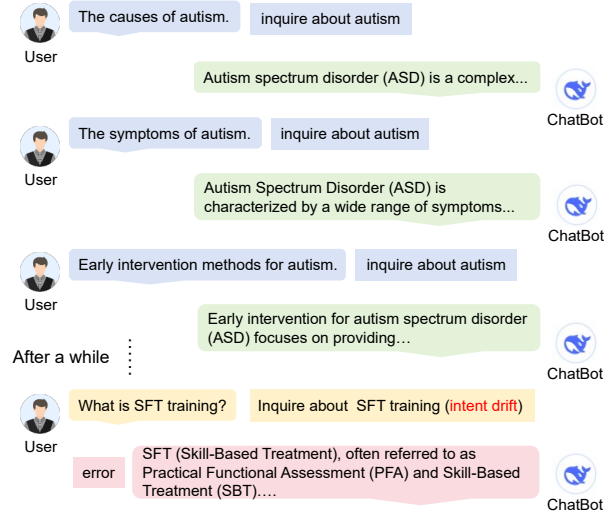


Fig. 1: An example of a potential error in real-world LLM-driven chatbots arises from intent drift.

Figure 1 presents a motivating example illustrating how users often engage in continuous, free-flowing conversations with LLM-driven chatbots, frequently shifting between varying intents. Initially, the user maintains a consistent intent across the chatbot's first three questions. However, over time, the same session may be repurposed to inquire about a different topic—a phenomenon known as *intent drift*. This shift poses significant challenges for models that rely heavily on the full dialogue history, as it can introduce misleading

[†]Lu Bai is the corresponding author (email: bailu@bnu.edu.cn).

¹The source code and dataset are available at <https://github.com/FDHTJ/DriftHunter>

context. For instance, a user might input "SFT training" in a machine learning context, but the chatbot could incorrectly respond with information about physiology. These dynamic and implicit transitions highlight the need for models that can robustly detect intent and manage context.

Prior intent classification work in multi-turn QA systems [6] employed recurrent neural network (RNN) architectures to model contextual dependencies between adjacent timesteps, thereby capturing intent drift patterns. To better leverage contextual information, some approaches incorporated latent features from fixed max-length conversation contexts for intent classification [7]. However, these methods fail to fully address the latent relationships between intents and slots in conversational contexts, making them struggle with handling arbitrary intent drifts. More recently, heterogeneous network-based methods [11] have provided new insights into the problem of intent drift in continuous conversations. These approaches improve performance in continuous conversational contexts by constructing a global heterogeneous label graph that captures intent-slot transition relationships. Meanwhile, recent LLM-driven chat systems [1], with their ability to process long sequences, also show potential in managing standalone continuous conversations. However, existing methods [11], [12] disregard the temporal transition information across conversation turns, leading to performance degradation in long, continuous conversations with arbitrary intent drifts.

To address this challenge, we propose a differentiable module, DriftHunter, that detects the intent drifts via temporal transition accumulation. This module is designed to capture the temporal dynamics of past conversational context by discovering and accumulating transition patterns between intents and slots in user utterances as the conversation progresses. Through this mechanism, DriftHunter efficiently constructs a global joint label (with both intent and slot labels) structure that represents the statistical relationships between intents and slots. The accumulated transition structure not only captures sequential patterns but also updates dynamically as new user utterances are introduced. Therefore, our proposed module enables neural network models to achieve better performance across multiple tasks, including intent drift detection, traditional intent identification, and slot-filling. Unlike existing methods that require an offline process to construct a global statistical structure before training, our approach supports end-to-end training. This enables the model to simultaneously learn global transitions between intents and slots, and the sequentially evolving patterns across conversational turns. Consequently, our approach is highly effective for large-scale QA systems serving multiple concurrent users. It can function as an online switching mechanism, dynamically adjusting the range of context to optimize performance.

We conduct comprehensive experiments on real-world public datasets. The experimental results show that our prototype system significantly outperforms state-of-the-art baselines across intent drift detection, intent identification, and slot-filling tasks. Applied to existing models, DriftHunter boosts their average F_1 scores by 6.4% in intent drift detection,

4.6% in intent detection, and 7.3% in slot filling. This verifies that our proposed DriftHunter effectively captures global heterogeneous information within user utterances in continuous conversations. In our case study, analysis of DriftHunter's trained parameters showed that the learned patterns accurately capture ground-truth user intents and reflect intent drift. This highlights our method's efficacy in identifying high-quality, temporally evolving patterns in QA interactions.

Contributions. In summary, the primary contributions of this paper are as follows:

- To the best of our knowledge, we first formulate the intent drift detection task. We also discuss its related challenges in a standalone and continuous conversation scenario.
- We propose a module DriftHunter, designed to end-to-end accumulate heterogeneous information between intents and slots as the conversation progresses. This is the first differentiable method to accumulate the temporal transition dynamics between intents and slots across different turns.
- We verify that by integrating DriftHunter, neural network models could achieve better performance on both the intent drift detection, intent identification, and slot filling tasks. Furthermore, the accumulated transitions observed in the case study demonstrate the proposed model's ability to align with the ground-truth user intents promptly. This further indicates that DriftHunter effectively captures the actual temporal dynamics underlying intent drifts.

II. RELATED WORKS

Spoken Language Understanding (SLU) [9] is crucial for dialogue systems, focusing on extracting structural semantic information from user utterances. Key SLU sub-tasks include intent detection [13], slot filling [14], and domain classification [15]. Current SLU models commonly employ an encoder-decoder framework and are broadly categorized as task-specific or joint models [16]. Joint models utilize a multi-task framework to address sub-tasks concurrently. Since joint models can leverage the hidden information between sub-tasks to achieve better performance on each, we also apply this framework in our work.

Intent identification, also known as intent classification [13], aims to determine the user's intention from their utterance. Slot filling [14] is a related task concerned with extracting relevant arguments (slots) pertinent to a detected intent within natural language utterances. Recognizing the inherent connection between these tasks, recent research has increasingly explored joint intent detection and slot filling models [9] under the multitask learning framework. Since [17], the self-attention mechanism has proven highly effective within the field of natural language processing. The attention mechanism has been widely adopted in intent detection and slot filling [12], [18] to enhance the semantic representation.

However, most of these methods are designed for single-turn conversational contexts and struggle to address the complexities of continuous and multi-turn QA scenarios.

With the advancement of LLM-driven chatbot systems, users are increasingly accustomed to continuous single-session

conversations [4]. Therefore, more and more researches focus on the intent identification problem in multi-turn or continuous conversation [19]. The primary consideration for intent identification in multi-turn conversations is the approach taken to leverage the contextual information available across the dialogue. Pioneering work by Mensio et al. [6] aided subsequent intent identification by concatenating contexts from adjacent time steps. Building upon this, Tran et al. [7] applied static, complete session contexts to further enhance intent classification. More recently, Xing et al. [11] discovered the potential of the relationship between intents and slots to improve intent identification performance in multi-turn continuous conversations. Consequently, Zhu et al. [12] employed a statistical approach to capture the global relationships between intents and slots, thereby improving the model's understanding of the context and leading to further performance gains. However, despite the benefits of leveraging statistical relationships between intents and slots, the current method experiences performance degradation when intents drift arbitrarily in continuous and standalone conversations.

III. PRELIMINARIES

A. Basic Notations

In QA or chatbot systems, users interact by entering short text messages or utterances, and receive responses that contain the requested information. Therefore, accurately determining the user's intent is the crucial first step in generating suitable answers. We give the formal definitions for the related notations by following the previous works [12].

Intent and Slot of User Utterance. Given a user utterance u , its intent is represented by a class label c , where c belongs to a predefined set of intent classes $\mathcal{C}$. Intent c reflects the underlying goal or purpose of u ; the slots of u are several pieces of u that provides specific details related to the user's intent. For instance, when the input is "The causes of autism," the system identifies the intent as "inquire about illness," and extracts the following slots: "illness name: autism" and "information type: causes."

Continuous and Standalone Chat System. A continuous and standalone chat system maintains a persistent session. This allows for uninterrupted user interaction. Unlike traditional systems, it does not necessitate manual intent switching for subsequent utterances. e.g., LLM-driven chatbot systems often include session switch buttons. However, most users tend to overlook these. They interact with the system continuously, treating it like a persistent human.

B. Problem Definition

Intent drift occurs in the following scenario: a user might initially stick to an intent due to unsuitable chatbot responses or the need for more relevant information. However, within the same session, they may later change their intent when a new requirement arises. We formally define intent drift and the problem of its detection as follows.

Intent Drift. In a QA system, let $U = (u_1, u_2, u_3, \dots, u_n)$ be a sequence of user utterances within a single session. Suppose $u_1, u_2, u_3, \dots, u_k$ ($k < n - 1$) have the intent c_k , and u_{k+1} relates to an intent c' , where $c' \neq c_k$, then an intent drift occurs at the k -th turn of the conversation, denoted as $o_k = 1$ (indicating a drift) and $o_k = 0$ otherwise.

We observe that the intent drift occurs arbitrarily in the QA systems. As introduced earlier, users accustomed to LLM-driven chatbots may provide additional information for a prior intent due to factors such as unsatisfactory responses or the need for more relevant information. This habit of omitting manual session switching can lead to arbitrary intent shifts when new requirements arise. To this end, we provide the following definition for the intent drift detection problem.

Intent Drift Detection in Continuous Conversation. With the same setting of intent drift, the intent drift detector in continuous conversation is a binary classifier. It is defined as the following function.

$$g_\theta(u_1, u_2, \dots, u_k, u_{k+1}) \rightarrow \{0, 1\}, \quad (1)$$

where g_θ denotes a model parameterized by θ that predicts the intent drift at the k -th conversational turn using the context from u_1 to u_k . Then, the intent drift detection is a task of optimizing θ to minimize the following equation.

$$\sum_{k=1,2,\dots,n} \mathcal{L}(o_k, g_\theta(u_1, u_2, \dots, u_k, u_{k+1})), \quad (2)$$

where $\mathcal{L}$ represents any suitable loss function that quantifies the discrepancy between the predicted label and the ground-truth o_k for the intent drift at the k -th conversational turn.

This problem can be treated as an extension of intent classification. Detected intent drifts can serve as crucial signals for chatbot systems, prompting improvements in intent classification and slot-filling performance. Existing intent classification methods statistically capture static, global relationships between intents and slots, but overlook dynamic temporal relationships across conversational turns that indicate the intent drift patterns. Neglecting this temporal aspect introduces a bias between the overall conversational context and the recent utterance, posing a significant challenge to accurately predict intents in a continuous conversation with intent drifts.

IV. PROPOSED MODEL

A. An Overview

We adopt a typical encoder-decoder multitask architecture for SLU [9] to address intent drift detection, intent classification, and slot filling tasks jointly. The overall framework of the proposed model is depicted in Figure 2. As shown in Figure 2, we propose DriftHunter, a general module situated between the encoder and decoder. The DriftHunter module involves four key steps: local representation learning, intent-slot transition accumulation, global transition incorporation, and token-level enhanced embedding.

First, local representation learning captures per-utterance representations. Next, intent-slot transition accumulation is

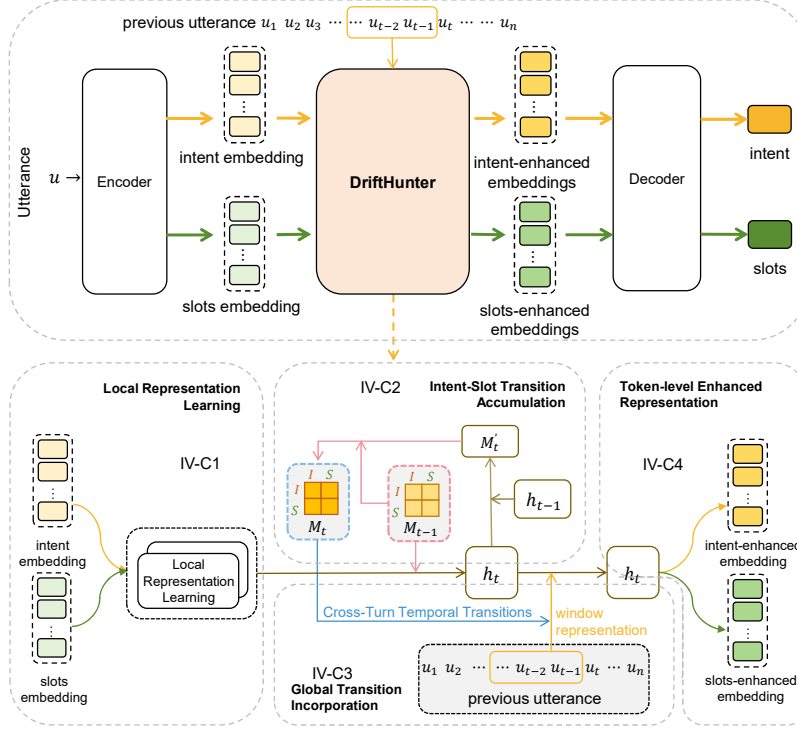


Fig. 2: The framework for DriftHunter. It comprises four key components: local representation learning, intent-slot transition accumulation, global transition incorporation, and token-level enhanced representation. Each of these will be detailed in Sections IV-C1, IV-C2, IV-C3, and IV-C4, respectively.

the critical step, learning dynamic temporal transition information across conversational turns. This capability significantly enhances performance in continuous dialogue scenarios. Following this, global transition incorporation integrates the information from the previous utterance into the local representations. Finally, token-level enhanced embedding leverages these results for downstream tasks. The following subsections provide a detailed explanation of each step.

B. The Encoder-Decoder Architecture

Our encoder-decoder architecture first encodes user utterances into latent embedding vectors, which are then decoded to perform intent classification and slot filling. Formally, given a user utterance u , the encoder represents this input as the following combined vector:

$$e_u = e_i \oplus e_s, \quad (3)$$

where the intent embeddings e_i and slot embeddings e_s for each token in the utterance u are $\mathbb{R}^{|u| \times d}$ matrices, and $\oplus$ represents their concatenation.

C. The DriftHunter Model

Building upon the base embedding vectors for intents and slots, DriftHunter accumulates temporal information between conversational turns through the temporal transition accumulation. As shown in Figure 2, DriftHunter consists of four main components, namely: local representation learning, global

transition incorporation, intent-slot transition accumulation, and token-level enhanced representation.

1) *The Local Representation Learning*: The local representation learning module captures local, per-utterance representations within a single timestep. Building upon the work of [20], this involves a weighted sum of token-level and utterance-level representations for each utterance, where the weights are derived from learned attention mechanisms that consider both representation levels.

Token-level Representation. To derive the token-level representation for an utterance, we leverage the combined embedding vectors that capture information about both intents and slots, and define the token-level representation as follows:

$$h^t = \text{softmax}(W_2 \tanh(e_u^\top \cdot W_1))e_u, \quad (4)$$

where $W_1 \in \mathbb{R}^{d \times d}$ and $W_2 \in \mathbb{R}^{|I+S| \times d}$ denote trainable matrices, and d represents the latent feature dimension.

Utterance-level Representation. Since the semantic information in the entire utterance also plays a non-negligible role in determining the labels. Drawing inspiration from [18], we introduce the utterance-label attention mechanism to capture the relationship between utterance-level semantics and labels, thereby enriching the representation of utterances:

$$h^u = (e_u^\top \cdot W_3)e_u, \quad (5)$$

where $W_3 \in \mathbb{R}^{|\delta| \times d}$ is trainable parameters. Then, the h^t and h^u are merged by adaptive fusion [18]:

$$h = \alpha h^t + (1 - \alpha) h^u, \quad (6)$$

where α is obtained from:

$$\alpha = \frac{\sigma(h^t W_4)}{\sigma(h^t W_4) + \sigma(h^u W_5)}. \quad (7)$$

The σ denotes the Sigmoid function, and $W_4 \in \mathbb{R}^{d \times d}$ and $W_5 \in \mathbb{R}^{d \times d}$ are the parameters. Ultimately, we derive the joint label embedding h for both the intents and slots.

2) *The Intent-Slot Transition Accumulation*: Our model accumulates the global transition cues for intents and slots based on the local representation information derived from utterances. Concurrently, it also learns the temporal changing patterns of the transitions between different dialogue turns. To this end, a sequence of joint transition matrices is used to learn the basic transitions between intents and slots, operating within a shared label space. Subsequently, temporal transitions across different turns are learned based on the obtained transition.

Joint Transition Matrix. By combining the labels for intents and slots to a joint label space, the joint transition matrix for the context at the t -th timestep is denoted as:

$$M_t = \{m_{i,j}\}_{(|I|+|S|) \times (|I|+|S|)}, \quad (8)$$

where I and S are the label sets of intents and slots in the corresponding conversation, and $m_{i,j}$ denotes the transition frequency between i -th and j -th labels in the joint label space. We commence by leveraging the joint transition matrix M to obtain the current embedding enhanced by temporal transition:

$$h_t = M_{t-1}^T h W_\tau + h, \quad (9)$$

where h_t is the joint embedding enhanced by t -th transition, and $W_\tau \in \mathbb{R}^{2d \times d}$ is a trainable temporal latent feature accumulation throughout the continuous conversation.

Cross-Turn Temporal Transitions. With the joint transition matrices at different conversation turns, the temporal changes between these matrices define the cross-turn temporal transition, denoted as:

$$M'_t = \text{Activation}(\text{Similarity}(h_{t-1}, h_t)), \quad (10)$$

where the Similarity operation computes pairwise similarities between each vector in h_{t-1} and all vectors in h_t , and the Activation is used to turn the value into a specific range. To preserve all temporal dynamics while emphasizing the influence of the most recent transition matrix, we apply exponential smoothing to update M_t :

$$M_t = \lambda M_{t-1} + (1 - \lambda) M'_t, \quad (11)$$

where λ serves as a hyperparameter that balances the contribution of the cross-turn temporal transition and the updated joint transition matrix, which represents the weighting of historical information during the update process, M_t in Equation 11 is subsequently used for all further computations.

3) *The Global Transition Incorporation*: Our model leverages a sliding window [7] to collect historical information, including temporal transition patterns, from preceding utterances, thus obtaining a global representation for downstream tasks.

Utterance Representation in Sliding Window. Suppose $\mathbf{U} = (u_1, u_2, \dots, u_n)$ is a sequence of utterances within the latest window of size n . Then, the representation of these utterances in this window is denoted as:

$$\mathcal{E} = e_{u_1} \oplus e_{u_2} \dots \oplus e_{u_n}, \quad (12)$$

where $\oplus$ is the concatenation operator and each e_{u_i} ($i = 1, 2, 3, \dots, n$) corresponds to an utterance-level representation generated by the SLU encoder. Intuitively, $\mathcal{E}$ refers to the previous latent information of this dialogue. While we implement BERT [21] as our foundational encoder, the architecture is compatible with alternative pre-trained language models.

Transition Incorporation. Our model derives an initial global representation for the joint label space by leveraging the token-level attended representations.

$$\mathcal{H} = (W_1 \mathcal{E}^T) \mathcal{E}, \quad (13)$$

where $W_1^U \in \mathbb{R}^{n \cdot |I \cup S| \times d}$ is trainable parameter and d denotes the embedding dimension of tokens.

The transition information from the DriftHunter module is incorporated into the window representation as follows:

$$h_t = M_t^T \mathcal{H} W_2 \oplus h_t, \quad (14)$$

where $W_2 \in \mathbb{R}^{D \times D}$ is a trainable parameter that adjusts the weight for the predicted transition information, M_t is the enhanced joint transition matrix from Equation 11 and D is the dimension of the pretrained embedding.

4) *The Token-level Enhanced Representation*: Leveraging the globally learned representation from the continuous conversation, our model decodes it to obtain enhanced intent and slot embeddings for downstream tasks. Specifically, we enhance the utterance embedding vectors using the joint embedding vector h_t from the last timestep of the DriftHunter process (Equation 9), following [12].

$$e_u = \text{softmax}(e_u h_t^T) h_t + e_u. \quad (15)$$

We further incorporate neighborhood features of tokens [22] and apply a linear transformation to obtain the final embeddings for intents and slots used in downstream tasks.

$$e_i, e_s = \text{Adj}(e_u), \quad (16)$$

where the function Adj aggregates the embedding for each token with its adjacent tokens in a given utterance u .

Compatible with Downstream tasks. The obtained representations e_i and e_s are fed into the SLU decoder to produce the downstream task logits $\hat{o} \in \mathbb{R}^1$, the intent logits $\hat{o}^I \in \mathbb{R}^{|I|}$, and the slot logits $\hat{o}^S \in \mathbb{R}^{|u| \times |S|}$.

We treat intent drift as a special case of intent classification and therefore apply the same decoding process used for standard intent prediction.

D. The Training of the Proposed Model

We address intent drift detection as a binary classification, using the Binary Cross-Entropy (BCE) [23] with logit loss.

$$\mathcal{L}_{\text{Drift}} = -\frac{1}{N} \sum_{k=1}^N \text{BCE}(o_k, \hat{o}_k), \quad (17)$$

where the o_k is the ground-truth of k-th intent drifts.

For intent detection and slot filling, we apply Cross-Entropy loss [24]:

$$\mathcal{L}_{\text{Intent}} = -\frac{1}{N} \sum_{k=1}^N o_k^I \log(\text{softmax}(\hat{o}_k^I)), \quad (18)$$

$$\mathcal{L}_{\text{Slots}} = -\frac{1}{NT} \sum_{k=1}^N \sum_{t=1}^T o_k^S \log(\text{softmax}(\hat{o}_k^S)), \quad (19)$$

where o_k^I and o_k^S represent the one-hot encodings of the k-th true label, and T is the number of tokens in the single utterance. So the joint objective is:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{drift}} + \beta \mathcal{L}_{\text{intent}} + \gamma \mathcal{L}_{\text{slots}}, \quad (20)$$

where α, β, γ are positive hyperparameters ($\alpha, \beta, \gamma > 0$) that adjust the weights for each downstream task, subject to the constraint $\alpha + \beta + \gamma = 1$.

V. EXPERIMENTS AND DISCUSSIONS

We aim to address the following research questions (RQ).

- **RQ1:** Can the proposed DriftHunter enhance the ability of models to detect intent drift across different scenarios?
- **RQ2:** What are the specific uses of the four components of the proposed DriftHunter mechanism?
- **RQ3:** How sensitive is DriftHunter to different parameters?
- **RQ4:** How to explain the learned parameters?

A. Datasets

To thoroughly evaluate the effectiveness of our method, we have selected three distinct datasets that are widely utilized in the field of intent classification and slot filling:

- Airline Travel Information System (ATIS) [25] dataset is a collection of spoken queries and responses related to airline travel, such as flight reservations, flight status, and airport information. This dataset is not treated as a conversation dataset, and each sentence has an intent.
- The Simulated-Dialogue dataset (Sim) [26] is an open-source, multi-turn dialogue dataset encompassing various application scenarios, including restaurant reservations and movie ticket purchases. Each dialogue in the dataset is designed to reflect a single intent.
- Multi-Domain Wizard-of-Oz Dataset (MultiWOZ) [27] is a widely recognized task-oriented dialogue dataset comprising over 10,000 annotated conversations across 8 domains. It is extensively used as a benchmark for dialogue state tracking. Each sentence conveys a specific intent, and these intents may change between sentences throughout the dialogue.

The details of each dataset are listed in Table I, where the average intent drift length is the average length of continuous, same-intent utterances per dataset.

TABLE I: Details of the datasets

Dataset name	ATIS	SIM	Multiwoz
Sentences	5,866	23,357	128,296
Training sent.	4,978	14,796	113,552
Testing sent.	888	8561	14,744
Ave. intent drift len.	2.22	15.35	6.73

B. Experimental Dataset Usage

Experiments on original datasets. We evaluated models on the original SIM and MultiWOZ datasets, given their inherent multi-turn conversational structure. The ATIS dataset, containing only single-turn conversations, was excluded.

Experiments on long continuous conversations. We constructed three simulated datasets from all the original data to assess model performance across extended continuous conversations. To build these datasets, we first randomly shuffled the original dialogues, then concatenated them to create continuous conversations. Finally, we iterated through each user utterance, marking any as intent drift if its intent differed from the previous utterance.

TABLE II: The prompt for DeepSeek precedes the utterances.

DeepSeek-R1-8B Prompt

```
Previous utterance:
<previous utterance>
Current utterance:
<current utterance>
Instruction:
You are given a previous utterance and a current utterance from the user in a dialogue. Your task is to determine whether the intent of the current utterance is different from the previous one.
Response format:
Analysis:
Provide a step-by-step reasoning comparing the two utterances, focusing on intent differences.
Decision:
Output one of the following: [Yes, they have different intents./No, they have the same intent.]
```

C. Baselines

- BERT [5] employs a pre-trained BERT encoder fine-tuned for joint intent detection and slot filling. It formulates both tasks as token-level and sentence-level classification problems, respectively, using the shared contextualized representations from BERT.
- RNNContextual [6] models contextual information by leveraging recurrent neural networks (RNNs) across dialogue turns. It updates the current utterance representation by incorporating the information from previous turns to improve multi-turn spoken language understanding.
- AGLCF [7] introduces an adaptive global-local context fusion mechanism. It employs multi-head attention to separately model global dialogue-level and local utterance-level information, and then fuses them to enhance both intent detection and slot filling.
- DHLG [12] constructs dual heterogeneous label graphs to capture label dependencies. It builds a global static graph

TABLE III: Result on Original SIM

Model	Intent Drift Detection			Intent Detection			Slot Filling		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BERT [5]	61.4	71.0	65.9	80.5	78.2	78.9	95.7	92.7	93.7
RNNContextual [6]	95.7	96.3	96.0	97.7	97.4	97.5	-	-	-
AGLCF [7]	99.8	100.0	99.9	99.0	99.0	99.0	95.7	94.4	94.8
DHLG [12]	95.8	97.2	96.5	98.6	98.4	98.5	90.8	89.4	89.6
BERT-DH	99.5	99.8	99.7	99.4	99.3	99.3	95.0	94.0	94.3
RNNContextual-DH	99.7	99.7	99.7	99.8	99.8	99.8	97.3	96.0	96.5
AGLCF-DH	99.7	100.0	99.8	99.3	99.2	99.3	95.7	94.5	94.9
DHLG-DH	99.7	99.8	99.8	98.7	98.5	98.6	93.7	92.8	92.7

TABLE IV: Result on Original Multiwoz

Model	Intent Drift Detection			Intent Detection			Slot Filling		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BERT [5]	84.0	91.6	87.6	73.9	72.1	72.6	86.9	90.6	88.6
RNNContextual [6]	91.5	90.0	90.8	79.2	78.1	78.6	-	-	-
AGLCF [7]	90.0	94.0	92.0	78.2	78.7	78.0	87.0	93.8	90.2
DHLG [12]	91.2	91.7	91.5	93.1	86.1	88.6	87.2	88.9	87.8
DEEPSEEK-R1-8B [1]	45.7	96.5	62.0	-	-	-	-	-	-
BERT-DH	87.3	90.4	88.8	82.2	79.7	80.4	88.2	90.8	89.4
RNNContextual-DH	89.6	91.9	90.7	86.6	85.5	85.9	89.2	92.6	90.8
AGLCF-DH	92.7	91.9	92.3	85.9	86.7	85.8	88.5	93.0	90.6
DHLG-DH	91.6	91.8	91.7	94.0	86.1	89.0	90.3	93.0	91.6

based on dataset-wide co-occurrence statistics and a local dynamic graph conditioned on each input, enabling more effective interaction between intent and slot labels.

- DeepSeek [1]. Given the strong performance of LLM-driven chatbots across various conversational tasks, we deployed a local DeepSeek-R1-8B for downstream applications. To enable DeepSeek to handle our specific task, we precede the session utterances with the instruction or prompt in Table II.

Since RNNContextual was not designed for slot filling, we did not evaluate it on this task to preserve its original performance in intent detection. Furthermore, due to the large number of labels involved, applying DeepSeek to both intent detection and slot filling is challenging. Therefore, we only report the performance of DeepSeek on intent drift detection.

In the original SIM dataset, intent drift occurs only at the beginning of each dialogue. If we were to evaluate DeepSeek on the original SIM data, we would only be able to provide the initial utterance as input at the point of drift, which would result in an unfair evaluation. Therefore, we did not conduct experiments using DeepSeek on the original SIM dataset.

For performance evaluation, models enhanced by the proposed DriftHunter are denoted by appending the suffix '-DH'.

D. Implementation Details

For our experiments, the maximum number of tokens for each utterance is 128, and the window size of the previous utterance is 32. For the comparison tasks, we used Cosine similarity and Softmax as the Similarity and the Activation functions, respectively. We utilize the pretrained bert-based-uncased model of BERT [21] for embedding, resulting in a dimensionality (D) of 768. We set λ in Equation 11 to 0.2 for all experiments. The rationale behind this, along with the settings for other hyperparameters like α , β and γ , is detailed

in Section V-E3. We adopt AdamW [28] to train the model with the learning rate of $1e-5$, and we select the best performance on the test set of each model. All experiments are conducted on an Nvidia 4090 GPU, and we fix the random seed to ensure reproduction.

E. Results and Discussions

1) *Comparison on Downstream Tasks (RQ1)*: We first investigate the performance of DriftHunter on the original datasets with relatively long average intent drift lengths (SIM, MultiWOZ), where we train and test the model on each conversation separately. This comparison was conducted in a multitasking setting encompassing intent drift detection, intent detection, and slot filling. As shown in Table III and IV, the performance of the version with DriftHunter generally outperforms the version without DriftHunter across all subtasks for each baseline. This result indicates that our method has effectively learned to detect intent drift in real-world scenarios.

To further analyze the proposed method's performance on extremely long conversations, we compared it with baselines across all datasets in Table I by following the synthetic data construction steps detailed in V-B. The results are shown in Tables V, VI, and VII. As these tables show, our DriftHunter module achieves significant improvements across all three subtasks, particularly in intent drift detection. Moreover, the results from DeepSeek-RL-8B indicate that even large language models still struggle with understanding intent drift, aligning with the motivating example discussed earlier.

Notably, even with a similar global statistical framework for slots and intents, DHLG's slot filling performance on the ATIS dataset is relatively low, likely due to the dataset's limited size

TABLE V: Detailed comparison on ATIS testing set.

Model	Intent Drift Detection			Intent Detection			Slot Filling		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BERT [5]	59.3	58.9	59.1	93.3	93.1	91.7	81.3	79.3	78.3
RNNContextual [6]	61.6	55.9	58.5	92.6	93.1	91.2	-	-	-
AGLCF [7]	76.2	95.9	84.9	93.8	94.7	93.0	74.9	76.1	73.0
DHLG [12]	76.7	96.3	85.4	93.6	93.3	92.2	31.4	33.5	29.6
DEEPSEEK-R1-8B [1]	35.8	95.9	52.1	-	-	-	-	-	-
BERT-DH	60.0	60.0	60.0	94.3	94.2	92.9	84.0	82.0	81.2
RNNContextual-DH	58.5	61.1	59.8	95.2	94.4	93.5	82.7	80.9	80.0
AGLCF-DH	80.1	95.6	87.6	93.7	95.2	93.1	83.7	82.3	81.2
DHLG-DH	78.4	95.2	86.0	95.1	94.4	93.5	72.7	68.2	68.3

TABLE VI: Detailed comparison on SIM testing set.

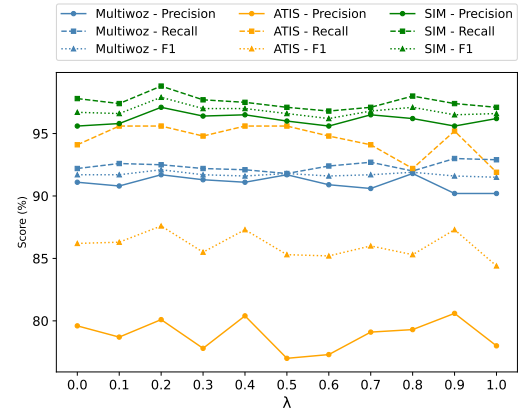
Model	Intent Drift Detection			Intent Detection			Slot Filling		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BERT [5]	59.0	76.0	66.4	81.7	78.7	79.6	94.3	92.2	92.8
RNNContextual [6]	97.1	31.0	47.0	52.5	66.3	57.5	-	-	-
AGLCF [7]	78.4	83.4	80.8	98.6	98.3	98.4	95.6	93.7	94.3
DHLG [12]	96.4	94.0	95.2	98.6	98.2	98.4	81.1	82.4	80.9
DEEPSEEK-R1-8B [1]	14.2	98.2	24.8	-	-	-	-	-	-
BERT-DH	59.7	78.8	67.9	83.7	81.1	82.0	95.6	93.5	94.1
RNNContextual-DH	97.2	95.9	96.5	97.6	97.2	97.4	97.1	94.7	95.7
AGLCF-DH	97.1	98.8	97.9	99.2	99.2	99.2	94.9	94.8	94.7
DHLG-DH	95.6	96.6	96.1	98.7	98.3	98.4	94.9	93.3	93.8

hindering its ability to capture accurate statistical information. In contrast, incorporating DriftHunter significantly improves DHLG-DH’s performance, suggesting that DriftHunter captures temporal latent information even with limited data.

Furthermore, it demonstrates that on datasets with longer contexts, the gains from DriftHunter are even greater, indicating that DriftHunter is particularly well-suited to handling long conversations.

2) *Ablation Study (RQ2)*: To evaluate the contribution of each component within our proposed module, we performed an study of the model across all three datasets. Given that AGLCF-DH achieved the best intent drift detection performance across all datasets, we selected it for this analysis. This ablation study was conducted by removing two key components of our DriftHunter module, intent-slot transition accumulation and global transition incorporation, across the three datasets. The results are reported in Table VIII. The results indicate that intent-slot transition accumulation primarily improves recall, whereas global transition incorporation enhances precision and F1-score. Together, these components lead to an overall performance gain.

To further investigate the contribution of intent drift detection to overall performance, we tested AGLCF-DH’s performance on intent detection and slot filling after removing the intent drift detection subtask. As Tables IX and X show, removing this module led to a noticeable drop in F1-score for both tasks. These results demonstrate that explicitly modeling intent drift enhances the model’s understanding of dialogue dynamics and improves performance.

Fig. 3: The sensitivity analysis of λ with AGLCF-DH.

3) *Parameter Sensitivity Analysis (RQ3)*: To optimize the performance of our proposed module, we evaluated its sensitivity to the hyperparameters of its components. Due to its strong performance, AGLCF-DH was chosen for a parameter sensitivity analysis across the three datasets. The analysis focuses on the hyperparameter λ in Equation 11, which determines the proportion of historical λ used during the update process. We varied λ from 0 to 1.0 in steps of 0.1. As shown in Figure 3, we select $\lambda = 0.2$ since it achieves a good balance across all datasets. Specifically, it delivers near-peak F1 scores on SIM and MultiWOZ, and maintains a competitive score on ATIS, showing the best trade-off between precision and recall in all scenarios.

For the hyperparameters α, β, γ in Equation 20, during our search for the best setting, we found that the more difficult the task, the higher the corresponding weight should be. Among the three tasks, ordered by difficulty, they are slot filling, intent

TABLE VII: Detailed comparison on Multiwoz testing set.

Model	Intent Drift Detection			Intent Detection			Slot Filling		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BERT [5]	83.8	91.7	87.5	75.6	72.7	73.9	85.5	91.1	88.1
RNNContextual [6]	90.4	91.7	91.1	87.1	85.2	86.1	-	-	-
AGLCF [7]	92.0	90.5	91.2	88.5	87.4	87.9	89.6	92.4	90.9
DHLG [12]	90.4	91.7	91.0	93.4	86.4	88.9	88.8	90.1	89.4
DEEPSEEK-R1-8B [1]	53.1	97.1	68.7	-	-	-	-	-	-
BERT-DH	83.2	92.2	87.5	83.0	79.6	81.0	87.4	90.7	88.9
RNNContextual-DH	91.2	91.2	91.2	87.3	85.1	86.1	88.2	92.3	90.2
AGLCF-DH	91.7	92.5	92.1	88.8	87.4	88.0	89.1	92.9	90.9
DHLG-DH	91.0	92.8	91.9	93.6	86.8	89.3	88.9	90.4	89.6

TABLE VIII: Impact of Removing Key Components

	Intent Drift Detection on ATIS			Intent Drift Detection on SIM			Intent Drift Detection on MULTWOZ		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
AGLCF-DH	80.1	95.6	87.6	97.1	98.8	97.9	91.7	92.5	92.1
<i>w/o</i> intent slot transition accumulation	77.7 (↓ 2.4)	94.4 (↓ 1.2)	85.3 (↓ 2.3)	96.0 (↓ 1.1)	96.0 (↓ 2.8)	96.0 (↓ 1.9)	91.6 (↓ 0.1)	91.7 (↓ 0.9)	91.6 (↓ 0.5)
<i>w/o</i> global transition incorporation	77.1 (↓ 3.0)	94.8 (↓ 0.8)	85.0 (↓ 2.6)	95.0 (↓ 2.1)	96.6 (↓ 2.2)	95.8 (↓ 2.1)	91.1 (↓ 0.6)	91.7 (↓ 0.8)	91.4 (↓ 0.7)

TABLE IX: Intent Detection Performance of Removing Intent Drift Task.

	Intent Detection on ATIS			Intent Detection on SIM			Intent Detection on MULTWOZ		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
AGLCF-DH	93.7	95.2	93.1	99.2	99.2	99.2	88.8	87.4	88.0
<i>w/o</i> intent drift detection	94.4 (↑ 0.7)	92.6 (↓ 2.6)	91.7 (↓ 1.4)	98.4 (↓ 0.8)	98.1 (↓ 1.1)	98.2 (↓ 1.0)	88.6 (↓ 0.2)	87.3 (↓ 0.1)	87.9 (↓ 0.1)

TABLE X: Slot Filling Performance of Removing Intent Drift Task.

	Slot Filling on ATIS			Slot Filling on SIM			Slot Filling on MULTWOZ		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
AGLCF-DH	83.7	82.3	81.2	94.9	94.8	94.7	89.1	92.9	90.9
<i>w/o</i> intent drift detection	62.5 (↓ 21.2)	58.7 (↓ 22.5)	57.9 (↓ 23.3)	96.5 (↑ 1.6)	91.7 (↓ 3.1)	93.2 (↓ 1.5)	89.3 (↑ 0.2)	92.1 (↓ 0.8)	90.6 (↓ 0.3)

detection, and intent drift detection. Therefore, we typically set (α, β, γ) to values like $(0.05, 0.25, 0.7)$. For different datasets and methods, the exact values should be adjusted accordingly. In general, α ranges from 0.05 to 0.15, β ranges from 0.2 to 0.35, and γ ranges from 0.6 to 0.8. The exact values can be found in our GitHub repository.

4) *Case Study (RQ4)*: To further investigate the dynamics of intent drift and maintenance, we visualized the intent transition matrix as a heatmap using our model on the SIM data. To enhance the visualization of differences, we employed dot similarity as Similarity and Tanh as Activation in Eq. 10. The analyzed cases were selected from the SIM test dataset using AGLCF-DH since it performs the best in intent drift detection across all datasets.

Figure 4 illustrates that as the user’s intent shifts from “buy movie tickets (BMT)” to “reserve restaurant (RR)”, there is an increasing trend in the corresponding cross-turn temporal transition. This indicates the DriftHunter effectively captures temporal dependencies between intents. The heatmaps also demonstrate the model’s ability to learn sequential user behaviors, such as planning dinner after buying movie tickets.

Figures 5 illustrates that when user intent remains consistent across interactions, the DriftHunter module effectively main-

tains high self-transition attention scores while suppressing irrelevant intent transitions. This indicates that the model effectively captures the temporal consistency of user behavior and avoids introducing noise from irrelevant intent paths. The ability to recognize and reinforce stable intent patterns highlights the DriftHunter module’s strength in modeling both dynamic drifts and steady states in multi-turn dialogues.

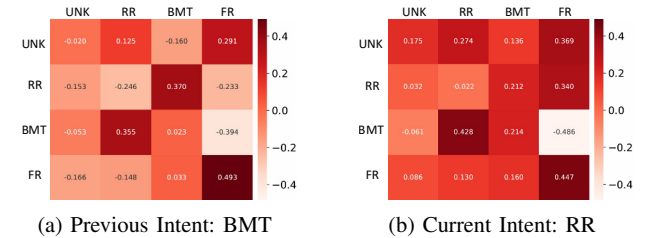


Fig. 4: Intent drifts from BMT to RR.

VI. CONCLUSION

In this paper, we address the prevalent intent drift problem in QA systems by first formalizing its definition. We then propose a neural network module, the DriftHunter, to tackle intent drift detection alongside intent detection and slot filling

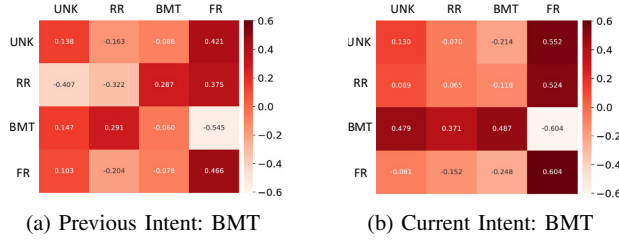


Fig. 5: Intent maintains “buy movie ticket (BMT)”.

within a multitasking framework. Since the proposed module is differentiable, it can be incorporated into any related neural network model to enhance its abilities when faced with the intent drift challenges. By employing a joint transition matrix and a cross-turn temporal transition structure, DriftHunter incrementally accumulates latent temporal transition information across different time windows, enabling it to achieve optimal performance even in long conversations. We conducted extensive experiments on public datasets and validated that the proposed DriftHunter module effectively enhances the performance of neural networks on intent drift detection, intent detection, and slot filling simultaneously.

VII. ACKNOWLEDGEMENTS

This work was supported by the National Natural Science Foundation of China (Grants 62576371 and T2122020) and the High-Performance Computing Platform of Central University of Finance and Economics.

REFERENCES

- [1] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [2] Z. Xu, M. J. Cruz, M. Guevara, T. Wang, M. Deshpande, X. Wang, and Z. Li, “Retrieval-augmented generation with knowledge graphs for customer service question answering,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 2905–2909.
- [3] C. Qian, B. He, Z. Zhuang, J. Deng, Y. Qin, X. Cong, Z. Zhang, J. Zhou, Y. Lin, Z. Liu *et al.*, “Tell me more! towards implicit user intention understanding of language model driven agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 1088–1113.
- [4] L. Yu, Z. Liu, L. Sun, B. Du, C. Liu, and W. Lv, “Continuous-time user preference modelling for temporal sets prediction,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 4, pp. 1475–1488, 2023.
- [5] Q. Chen, Z. Zhuo, and W. Wang, “Bert for joint intent classification and slot filling,” *arXiv preprint arXiv:1902.10909*, 2019.
- [6] M. Mensio, G. Rizzo, and M. Morisio, “Multi-turn qa: A rnn contextual approach to intent classification for goal-oriented systems,” in *Companion Proceedings of the The Web Conference 2018*, 2018, pp. 1075–1080.
- [7] T. Tran, K. Wei, W. Ruan, R. McGowan, N. Susanj, and G. P. Strimel, “Adaptive global-local context fusion for multi-turn spoken language understanding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 11, 2022, pp. 12 622–12 628.
- [8] Z. Sun, H. Liu, X. Qu, K. Feng, Y. Wang, and Y. S. Ong, “Large language models for intent-driven session recommendations,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 324–334.
- [9] H. Weld, X. Huang, S. Long, J. Poon, and S. C. Han, “A survey of joint intent detection and slot filling models in natural language understanding,” *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–38, 2022.
- [10] T. Kim, H. Yoon, Y. Lee, P. Kang, and M. Kim, “Mismatch between multi-turn dialogue and its evaluation metric in dialogue state tracking,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022, pp. 297–309.
- [11] B. Xing and I. Tsang, “Co-guiding net: Achieving mutual guidances between multiple intent detection and slot filling via heterogeneous semantics-label graphs,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 159–169.
- [12] Z. Zhu, X. Cheng, H. Li, Y. Li, and Y. Zou, “Dance with labels: Dual-heterogeneous label graph interaction for multi-intent spoken language understanding,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 1022–1031.
- [13] S. Cao, B. Qian, C. Yin, X. Li, J. Wei, Q. Zheng, and I. Davidson, “Knowledge guided short-text classification for healthcare applications,” in *2017 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2017, pp. 31–40.
- [14] G. Mesnil, Y. Dauphin, K. Yao, Y. Bengio, L. Deng, D. Hakkani-Tur, X. He, L. Heck, G. Tur, D. Yu *et al.*, “Using recurrent neural networks for slot filling in spoken language understanding,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 23, no. 3, pp. 530–539, 2014.
- [15] O. Agarwal and A. Nenkova, “Temporal effects on pre-trained models for language processing tasks,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 904–921, 2022.
- [16] L. Qin, T. Xie, W. Che, and T. Liu, “A survey on spoken language understanding: Recent advances and new frontiers,” in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021*, Z. Zhou, Ed. ijcai.org, 2021, pp. 4577–4584.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [18] M. Song, B. Yu, L. Quangan, W. Yubin, T. Liu, and H. Xu, “Enhancing joint multiple intent detection and slot filling with global intent-slot co-occurrence,” in *Proceedings of the 2022 conference on empirical methods in natural language processing*, 2022, pp. 7967–7977.
- [19] J. Liu, T. Keat, B. Fu, and K. H. Lim, “Lara: Linguistic-adaptive retrieval-augmentation for multi-turn intent classification,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, pp. 1096–1106.
- [20] L. Xiao, X. Huang, B. Chen, and L. Jing, “Label-specific document representation for multi-label text classification,” in *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 2019, pp. 466–475.
- [21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.
- [22] X. Zhang and H. Wang, “A joint model of intent determination and slot filling for spoken language understanding,” in *IJCAI*, vol. 16, no. 2016, 2016, pp. 2993–2999.
- [23] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [24] K. P. Murphy, *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [25] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington, “The atis spoken language systems pilot corpus,” in *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, June 24-27, 1990*, 1990.
- [26] P. Shah, D. Hakkani-Tür, G. Tür, A. Rastogi, A. Bapna, N. Nayak, and L. Heck, “Building a conversational agent overnight with dialogue self-play,” *arXiv preprint arXiv:1801.04871*, 2018.
- [27] X. Zang, A. Rastogi, S. Sunkara, R. Gupta, J. Zhang, and J. Chen, “Multiwoz 2.2: A dialogue dataset with additional annotation corrections and state tracking baselines,” in *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI, ACL 2020*, 2020, pp. 109–117.
- [28] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.